PADRÕES DE PROJETO: RESUMO DOS 22 PADRÕES CLÁSSICOS

Gustavo Silva dos Santos - Tecnologia em Análise e Desenvolvimento de Sistemas
Universidade Católica de Brasília (UCB) 2026

1 INTRODUÇÃO

Quando a gente desenvolve um sistema, é muito comum esbarrar em problemas que já apareceram em outros projetos e que já foram resolvidos de forma parecida por outros desenvolvedores. Os padrões de projeto (design patterns) são justamente isso: soluções típicas para problemas recorrentes no projeto de software orientado a objetos. Eles não são um código pronto nem uma biblioteca que se importa no projeto, mas sim um modelo de solução, que precisa ser adaptado ao contexto de cada sistema (GAMMA et al., 2000).

O conceito ficou conhecido com o livro de Gamma et al. (2000), que catalogou 23 padrões e popularizou o nome "Gang of Four" (GoF), em referência aos quatro autores. O catálogo do Refactoring.Guru (2026), que serve de base para este trabalho, organiza 22 desses padrões em três grupos, de acordo com o seu propósito: os criacionais, que tratam da criação de objetos; os estruturais, que mostram como montar classes e objetos em estruturas maiores; e os comportamentais, que cuidam dos algoritmos e da divisão de responsabilidades entre os objetos. O Quadro 1 resume os 22 padrões.

Quadro 1 - Os 22 padrões de projeto, suas categorias e seus propósitos

Padrão	Categoria	Propósito resumido
Factory Method	Criacional	Deixa as subclasses decidirem qual objeto será criado
Abstract Factory	Criacional	Cria famílias de objetos relacionados sem depender das classes concretas
Builder	Criacional	Constrói objetos complexos passo a passo
Prototype	Criacional	Cria novos objetos copiando um objeto existente
Singleton	Criacional	Garante uma única instância de uma classe, com acesso global
Adapter	Estrutural	Faz objetos com interfaces incompatíveis trabalharem juntos
Bridge	Estrutural	Separa a abstração da implementação em duas hierarquias
Composite	Estrutural	Trata objetos individuais e grupos de objetos da mesma forma, em árvore
Decorator	Estrutural	Adiciona comportamentos a um objeto sem alterar a sua classe
Facade	Estrutural	Oferece uma interface simples para um subsistema complexo
Flyweight	Estrutural	Economiza memória compartilhando partes comuns de muitos objetos
Proxy	Estrutural	Controla o acesso a um objeto por meio de um substituto

Padrão	Categoria	Propósito resumido
Chain of Responsibility	Comportamental	Passa um pedido por uma cadeia de manipuladores
Command	Comportamental	Transforma uma requisição em um objeto
Iterator	Comportamental	Percorre uma coleção sem expor sua estrutura interna
Mediator	Comportamental	Centraliza a comunicação entre objetos em um mediador
Memento	Comportamental	Salva e restaura o estado de um objeto
Observer	Comportamental	Notifica vários objetos quando algo muda em outro
State	Comportamental	Muda o comportamento do objeto conforme o seu estado interno
Strategy	Comportamental	Torna uma família de algoritmos intercambiável
Template Method	Comportamental	Define o esqueleto de um algoritmo e deixa as etapas para as subclasses
Visitor	Comportamental	Separa os algoritmos dos objetos sobre os quais eles operam

Fonte: adaptado de Refactoring.Guru (2026).

O objetivo deste trabalho é apresentar um resumo completo de cada padrão, seguindo sempre a mesma organização: nomes pelos quais o padrão é conhecido, intenção, problema, solução, estrutura, passo a passo de implementação, exemplo de código, quando usar, vantagens e desvantagens e a relação com os outros padrões. Os exemplos de código foram escritos em TypeScript, de forma simplificada, apenas para fins didáticos.

Alguns termos aparecem várias vezes ao longo do texto e vale explicar de antemão. O princípio da responsabilidade única (SRP, de Single Responsibility Principle) diz que uma classe deve ter apenas um motivo para mudar. O princípio aberto/fechado (OCP, de Open/Closed Principle) diz que o código deve estar aberto para extensão, mas fechado para modificação, ou seja, dá para adicionar funcionalidades novas sem mexer no que já funciona. Por fim, a preferência por composição em vez de herança significa montar o comportamento combinando objetos, e não apenas criando subclasses.

Para aproximar a teoria da prática, este trabalho também apresenta exemplos retirados do Adega Express, sistema de comércio eletrônico de bebidas não alcoólicas desenvolvido pelos autores no contexto da disciplina ministrada pelo professor. Nos padrões para os quais foram identificadas correspondências no código, foram incluídos trechos do próprio projeto, com identificação do arquivo de origem. As correspondências são classificadas conforme sua fidelidade ao padrão clássico; os exemplos não significam que todos os 22 padrões foram implementados no sistema.

2 PADRÕES CRIACIONAIS

Os padrões criacionais fornecem mecanismos para criar objetos, aumentando a flexibilidade e a reutilização do código que já existe (REFACTORING.GURU, 2026). A ideia geral é tirar do código cliente a responsabilidade de saber exatamente qual classe instanciar e como.

2.1 Factory Method

Também conhecido como: Método Fábrica, Construtor Virtual.

Intenção. Definir uma interface para criar objetos em uma superclasse, deixando que as subclasses decidam qual classe concreta será instanciada.

Problema. Imagine um sistema de logística que, no começo, só faz entregas por caminhão, e por isso todo o código foi escrito em cima da classe Caminhão. Quando surge a necessidade de entregar também por navio, quase tudo precisa ser alterado, e o código vai ficando cheio de condicionais para descobrir qual transporte usar.

Solução. Em vez de chamar o construtor diretamente (com o operador new), o código passa a chamar um método fábrica. Cada subclasse sobrescreve esse método e retorna um produto diferente. Para isso funcionar, todos os produtos precisam seguir uma interface comum, como Transporte com o método entregar. Assim, o cliente trabalha apenas com a interface e não precisa saber qual é o produto real.

Estrutura.

Produto: interface comum a todos os objetos criados.
Produtos concretos: implementações diferentes da interface do produto.
Criador: declara o método fábrica e, em geral, tem a lógica de negócio que usa os produtos.
Criadores concretos: sobrescrevem o método fábrica para retornar um tipo de produto específico.

Como implementar. (1) Fazer todos os produtos seguirem a mesma interface. (2) Adicionar um método fábrica na classe criadora. (3) Trocar as chamadas diretas ao construtor pelas chamadas ao método fábrica. (4) Criar uma subclasse do criador para cada tipo de produto e sobrescrever o método.

Exemplo de código. O Código 1 mostra o exemplo da logística.

Código 1 - Exemplo de Factory Method em TypeScript

```
interface Transporte {
  entregar(): string;
}
class Caminhao implements Transporte {
  entregar() { return "Entrega por terra"; }
}
class Navio implements Transporte {
  entregar() { return "Entrega por mar"; }
}
abstract class Logistica {
  abstract criarTransporte(): Transporte; // método fábrica
  planejarEntrega(): string {
    const transporte = this.criarTransporte();
    return transporte.entregar();
  }
}
class LogisticaRodoviaria extends Logistica {
  criarTransporte() { return new Caminhao(); }
}
class LogisticaMaritima extends Logistica {
  criarTransporte() { return new Navio(); }
}
console.log(new LogisticaMaritima().planejarEntrega()); // Entrega por
mar
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando não se sabe de antemão os tipos exatos dos objetos com que o código vai trabalhar; quando se quer permitir que quem usa uma biblioteca ou framework estenda os seus componentes internos; e quando se quer reaproveitar objetos que já existem, como conexões com o banco de dados, em vez de recriá-los toda hora.

Vantagens e desvantagens. O padrão evita o acoplamento forte entre o criador e os produtos concretos, concentra o código de criação em um único lugar (SRP) e permite incluir novos produtos sem quebrar o que já existe (OCP). Em contrapartida, o código pode ficar mais complicado, já que o padrão exige a criação de muitas subclasses novas.

Relação com outros padrões. Muitos projetos começam com o Factory Method, por ser mais simples, e evoluem para o Abstract Factory, o Prototype ou o Builder. Ele também pode ser visto como uma especialização do Template Method.

2.2 Abstract Factory

Também conhecido como: Fábrica Abstrata, Kit.

Intenção. Permitir a criação de famílias de objetos relacionados sem especificar as suas classes concretas.

Problema. Imagine um sistema de loja de móveis com uma família de produtos relacionados (cadeira, sofá e mesa de centro) disponível em estilos diferentes, como moderno e vitoriano. O cliente precisa receber móveis que combinem entre si, já que um sofá moderno não combina com uma cadeira vitoriana. Além disso, adicionar novos estilos ou novos produtos não deveria obrigar a mexer em código que já funciona.

Solução. O primeiro passo é declarar uma interface para cada tipo de produto da família (Cadeira, Sofá, Mesa) e fazer todas as variações implementarem essas interfaces. Depois, declara-se a fábrica abstrata, que tem um método de criação para cada produto, e cria-se uma fábrica concreta para cada estilo. O cliente recebe a fábrica pela interface abstrata, então não sabe qual estilo está usando, mas tem a garantia de que os produtos criados pelas mesma fábrica são compatíveis.

Estrutura.

Produtos abstratos: interfaces dos produtos distintos, mas relacionados, da família.
Produtos concretos: implementações dos produtos abstratos, agrupadas por variação.
Fábrica abstrata: interface com um método de criação para cada produto abstrato.
Fábricas concretas: cada uma cria apenas os produtos de uma variação.
Cliente: usa as fábricas e os produtos somente pelas interfaces abstratas.

Como implementar. (1) Mapear a matriz de tipos de produto por variação. (2) Declarar interfaces dos produtos abstratos. (3) Declarar a interface da fábrica abstrata. (4) Implementar uma fábrica concreta para cada variação. (5) Criar o código de inicialização que escolhe a fábrica e a passa ao cliente.

Exemplo de código. O Código 2 usa um aplicativo com tema claro e tema escuro, em que os componentes de cada tema não podem ser misturados.

Código 2 - Exemplo de Abstract Factory em TypeScript

```
interface Botao { desenhar(): string; }
interface CampoTexto { desenhar(): string; }
interface FabricaDeTema {
  criarBotao(): Botao;
  criarCampoTexto(): CampoTexto;
}
class BotaoClaro implements Botao {
  desenhar() { return "Botão com fundo branco"; }
}
class CampoTextoClaro implements CampoTexto {
  desenhar() { return "Campo de texto com fundo branco"; }
}
class BotaoEscuro implements Botao {
  desenhar() { return "Botão com fundo preto"; }
}
class CampoTextoEscuro implements CampoTexto {
  desenhar() { return "Campo de texto com fundo preto"; }
}
class FabricaTemaClaro implements FabricaDeTema {
  criarBotao() { return new BotaoClaro(); }
  criarCampoTexto() { return new CampoTextoClaro(); }
}
class FabricaTemaEscuro implements FabricaDeTema {
  criarBotao() { return new BotaoEscuro(); }
  criarCampoTexto() { return new CampoTextoEscuro(); }
}
function montarTela(fabrica: FabricaDeTema) {
  console.log(fabrica.criarCampoTexto().desenhar());
  console.log(fabrica.criarBotao().desenhar());
}
montarTela(new FabricaTemaEscuro()); // troca por FabricaTemaClaro e
tudo muda junto
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando o código precisa trabalhar com várias famílias de produtos relacionados, sem depender das classes concretas deles; e quando uma classe tem um conjunto de métodos fábrica que atrapalha a sua responsabilidade principal, caso em que vale extraí-los para uma fábrica própria.

Vantagens e desvantagens. O padrão garante que os produtos de uma mesma fábrica são compatíveis, evita o acoplamento entre produtos concretos e o cliente, e respeita o SRP e o OCP. A desvantagem é que o código fica mais complexo, por causa da quantidade de interfaces e classes novas, e adicionar um novo tipo de produto obriga a alterar todas as fábricas.

Relação com outros padrões. O Abstract Factory costuma ser baseado em um conjunto de Factory Methods. Diferente do Builder, que constrói um objeto complexo passo a passo, ele cria famílias de objetos e retorna o produto na hora. Pode ser uma alternativa ao Facade quando só se quer esconder do cliente a forma como os objetos do subsistema são criados, e as fábricas abstratas podem ser implementadas como Singletons.

Diferença entre Factory Method e Abstract Factory. O Factory Method cria um tipo de produto por meio de herança, enquanto o Abstract Factory cria uma família inteira de produtos por meio de composição, já que o cliente recebe um objeto fábrica. Na prática, se o que varia é um único tipo de objeto, o Factory Method resolve. Se vários objetos precisam variar juntos, o Abstract Factory é a melhor escolha.

2.3 Builder

Também conhecido como: Construtor.

Intenção. Construir objetos complexos passo a passo, permitindo produzir tipos e representações diferentes do objeto usando o mesmo código de construção.

Problema. Imagine um objeto complexo, com muitos campos e objetos aninhados, cuja inicialização exige um construtor gigante com dezenas de parâmetros, a maioria deles opcionais. Uma casa, por exemplo, pode ter ou não piscina, garagem, jardim e aquecimento. Criar uma subclasse para cada combinação gera uma enxurrada de classes, e usar um construtor único deixa as chamadas confusas, cheias de valores nulos. Esse problema é conhecido como "construtor telescópico".

Solução. O código de construção é retirado da classe do produto e colocado em objetos separados, chamados construtores (builders). Eles oferecem etapas como construir paredes ou construir telhado, e o cliente executa somente as etapas de que precisa. Opcionalmente, uma classe diretora (Director) define a ordem em que as etapas são chamadas, o que permite reaproveitar receitas de construção prontas. O mesmo processo pode, inclusive, gerar produtos diferentes, como uma casa e o manual de instruções dela.

Estrutura.

Interface do construtor: declara as etapas de construção comuns a todos os construtores.
Construtores concretos: implementam as etapas e guardam o produto que está sendo montado. Produto: o objeto resultante, que pode ter classes diferentes para cada construtor. Diretor: define a ordem de chamada das etapas (opcional). Cliente: associa um construtor ao diretor e recebe o produto pronto.

Como implementar. (1) Definir as etapas comuns de construção. (2) Declará-las na interface do construtor. (3) Criar um construtor concreto para cada representação do produto. (4) Pensar em criar uma classe diretora para as sequências de construção mais usadas. (5) Fazer o cliente criar o construtor, passá-lo ao diretor (ou chamar as etapas direto) e buscar o resultado.

Exemplo de código. O Código 3 mostra um construtor de pedidos de lanche, em que cada item é opcional.

Código 3 - Exemplo de Builder em TypeScript

```
class Lanche {
  itens: string[] = [];
  descricao() { return this.itens.join(", "); }
}
class LancheBuilder {
  private lanche = new Lanche();
  addPao(tipo: string) { this.lanche.itens.push(`pão ${tipo}`); return
this; }
  addHamburguer() { this.lanche.itens.push("hambúrguer"); return this; }
  addQueijo() { this.lanche.itens.push("queijo"); return this; }
  addBacon() { this.lanche.itens.push("bacon"); return this; }
  build(): Lanche { return this.lanche; }
}
const lanche = new LancheBuilder()
  .addPao("australiano")
  .addHamburguer()
  .addQueijo()
  .build();
console.log(lanche.descricao()); // pão australiano, hambúrguer, queijo
```

Fonte: elaborado pelo autor (2026).

Quando usar. Para se livrar do construtor telescópico; quando o código precisa criar diferentes representações do mesmo produto (como casas de madeira e de pedra); e para construir árvores de objetos, como as do padrão Composite, de forma controlada.

Vantagens e desvantagens. É possível construir objetos em etapas, adiar etapas ou executá-las recursivamente, reaproveitar o mesmo código para representações diferentes e isolar o código complexo de construção da lógica de negócio (SRP). Como ponto negativo, a complexidade geral aumenta, pois o padrão exige a criação de várias classes novas.

Relação com outros padrões. Muitos projetos começam com Factory Method e evoluem para Builder quando a criação fica mais complexa. O Builder pode ser implementado como Singleton e costuma ser usado para montar árvores Composite.

2.4 Prototype

Também conhecido como: Protótipo, Clone.

Intenção. Permitir copiar objetos existentes sem que o código fique dependente das classes deles.

Problema. Suponha que seja preciso criar uma cópia exata de um objeto. Copiar campo por campo, de fora do objeto, nem sempre é possível, porque alguns campos podem ser privados. Além disso, o código talvez conheça só a interface do objeto, e não a sua classe concreta, então nem saberia qual classe instanciar para fazer a cópia.

Solução. O padrão delega o processo de clonagem para o próprio objeto que está sendo clonado. Todas as classes clonáveis implementam uma interface comum com um método clone, que cria um novo objeto da mesma classe e copia para ele os valores do original. Um objeto que já está configurado e que pode ser clonado é chamado de protótipo, e é possível manter um registro de protótipos com as configurações mais usadas.

Estrutura.

Interface do protótipo: declara o método de clonagem.
Protótipo concreto: implementa o método de clonagem, copiando também os objetos internos quando necessário.
Cliente: produz uma cópia de qualquer objeto que siga a interface.
Registro de protótipos (opcional): guarda os protótipos mais usados para serem recuperados por nome ou categoria.

Como implementar. (1) Criar a interface do protótipo com o método clone. (2) Implementar o método em cada classe, tomando cuidado com os campos que são referências a outros objetos. (3) Opcionalmente, criar um registro de protótipos para acesso rápido às configurações prontas.

Exemplo de código. O Código 4 clona um personagem de jogo, copiando também o inventário para a cópia não compartilhar a mesma lista com o original.

Código 4 - Exemplo de Prototype em TypeScript

```
interface Clonavel<T> {
  clonar(): T;
}
class Personagem implements Clonavel<Personagem> {
  constructor(
    public nome: string,
    public vida: number,
    public inventario: string[]
  ) {}
  clonar(): Personagem {
    // cópia profunda: o inventário também é copiado
    return new Personagem(this.nome, this.vida, [...this.inventario]);
  }
}
const modelo = new Personagem("Goblin", 100, ["adaga"]);
const goblin2 = modelo.clonar();
goblin2.inventario.push("poção");
console.log(modelo.inventario); // ["adaga"]
console.log(goblin2.inventario); // ["adaga", "poção"]
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando o código não deve depender das classes concretas dos objetos que precisa copiar, e quando se quer reduzir o número de subclasses que só existem para configurar objetos de formas diferentes, trocando-as por protótipos já configurados.

Vantagens e desvantagens. É possível clonar objetos sem acoplamento às suas classes, eliminar código repetitivo de inicialização e produzir objetos complexos com mais conveniência. Por outro lado, clonar objetos complexos que têm referências circulares pode ser bem trabalhoso, e é preciso decidir entre a cópia rasa (só os valores do nível principal) e a cópia profunda (inclui os objetos internos).

Relação com outros padrões. O Prototype pode ser uma alternativa ao Factory Method e ao Abstract Factory quando se quer evitar hierarquias de subclasses. Também ajuda a guardar o histórico de comandos no Command e pode ser combinado com o Memento, embora este costume ser mais simples.

2.5 Singleton

Também conhecido como: Instância Única.

Intenção. Garantir que uma classe tenha apenas uma instância e fornecer um ponto de acesso global a ela.

Problema. O padrão resolve dois problemas ao mesmo tempo. O primeiro é garantir que exista um único objeto de uma classe, por exemplo uma conexão com o banco de dados ou um arquivo de configuração, em que várias instâncias poderiam gerar inconsistências. O segundo é oferecer um ponto de acesso global a essa instância, de forma mais segura do que uma simples variável global, que qualquer parte do código poderia sobrescrever.

Solução. A implementação costuma seguir dois passos. Primeiro, o construtor padrão se torna privado, para impedir que outros objetos usem o operador new. Depois, cria-se um método estático de criação (geralmente chamado getInstance), que cria o objeto na primeira chamada e guarda esse objeto em um campo estático, retornando sempre o mesmo nas chamadas seguintes.

Estrutura.

Singleton: declara o método estático que retorna a instância única e mantém o construtor privado.
Cliente: acessa a instância sempre pelo método estático.

Como implementar. (1) Adicionar um campo estático privado para guardar a instância. (2) Declarar o método estático público que a retorna, criando-a se for preciso. (3) Tornar o construtor privado. (4) Substituir as chamadas diretas ao construtor, no código cliente, pelas chamadas ao método estático.

Exemplo de código. O Código 5 mostra uma classe de configuração com instância única.

Código 5 - Exemplo de Singleton em TypeScript

```
class Configuracao {
  private static instancia: Configuracao | null = null;
  private constructor(public ambiente: string) {}

  static getInstance(): Configuracao {
    if (!Configuracao.instancia) {
      Configuracao.instancia = new Configuracao("producao");
    }
    return Configuracao.instancia;
  }
}
const a = Configuracao.getInstance();
const b = Configuracao.getInstance();
console.log(a === b); // true: é o mesmo objeto
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando uma classe do programa deve ter uma única instância disponível para todos os clientes, como um objeto de acesso ao banco de dados compartilhado, e quando se precisa de um controle mais rígido sobre as variáveis globais.

Vantagens e desvantagens. Há a garantia de uma única instância, o acesso global a ela e a inicialização apenas quando o objeto é pedido pela primeira vez. Mas o padrão é bastante criticado: ele viola o SRP, porque resolve dois problemas ao mesmo tempo; pode mascarar um projeto ruim, em que os componentes sabem demais uns dos outros; exige tratamento especial em ambientes com várias threads, para que duas delas não criem instâncias diferentes ao mesmo tempo; e dificulta os testes unitários, já que o construtor é privado e muitas vezes não dá para trocar a instância por uma versão falsa. Por isso, muitos autores o consideram um antipadrão quando é usado em excesso.

Relação com outros padrões. As classes de Facade muitas vezes podem virar Singletons, já que um único objeto de fachada costuma bastar. O Flyweight pode se parecer com o Singleton quando se reduz tudo a uma instância compartilhada, mas há duas diferenças: o Singleton tem só uma instância, enquanto o Flyweight pode ter várias com estados diferentes, e o objeto do Singleton pode ser mutável, enquanto os do Flyweight são imutáveis. Fábricas abstratas, Builders e Prototypes também podem ser implementados como Singletons.

3 PADRÕES ESTRUTURAIS

Os padrões estruturais explicam como montar objetos e classes em estruturas maiores, mantendo essas estruturas flexíveis e eficientes (REFACTORING.GURU, 2026). Em geral, eles lidam com a forma como as classes se relacionam por meio de herança e, principalmente, de composição.

3.1 Adapter

Também conhecido como: Adaptador, Wrapper.

Intenção. Permitir que objetos com interfaces incompatíveis trabalhem juntos.

Problema. Imagine uma aplicação que trabalha com dados em JSON e precisa usar uma biblioteca ou um sistema legado que só fornece os dados em XML. Alterar a biblioteca nem sempre é possível (o código pode ser de terceiros) e alterar a aplicação para falar a língua da biblioteca deixaria o código dependente de um formato específico.

Solução. Cria-se um adaptador, um objeto especial que converte a interface de um objeto para que outro objeto consiga entendê-lo. O adaptador implementa a interface que o cliente espera e guarda uma referência ao objeto adaptado. Quando o cliente chama o adaptador, ele converte os dados para o formato necessário, repassa a chamada ao objeto adaptado e devolve o resultado. É a mesma ideia de um adaptador de tomada, que permite ligar um plugue de um padrão em uma tomada de outro.

Estrutura. Existem duas versões do padrão:

Adaptador de objeto: usa composição, ou seja, o adaptador guarda uma referência ao objeto adaptado. É a versão mais comum.
Adaptador de classe: usa herança múltipla para herdar as duas interfaces ao mesmo tempo, e por isso só é possível em linguagens que a suportam.

Os participantes são o cliente (a classe com a lógica de negócio), a interface do cliente (o protocolo que o cliente espera), o serviço (a classe útil, mas incompatível) e o adaptador (que faz a ponte entre os dois).

Como implementar. (1) Identificar as duas classes com interfaces incompatíveis. (2) Declarar a interface do cliente. (3) Criar a classe adaptadora seguindo essa interface, com um campo para guardar a referência ao serviço. (4) Implementar os métodos da interface, convertendo os dados e delegando ao serviço. (5) Fazer o cliente usar o adaptador pela interface.

Exemplo de código. O Código 6 adapta um sistema legado que devolve XML para um cliente que só entende JSON.

Código 6 - Exemplo de Adapter em TypeScript

```
interface FonteJson {
  obterJson(): string;
}
class SistemaLegadoXml {
  obterXml(): string {
    return "<usuario><nome>Ana</nome></usuario>";
  }
}
class AdaptadorXmlParaJson implements FonteJson {
  constructor(private legado: SistemaLegadoXml) {}
  obterJson(): string {
    const xml = this.legado.obterXml();
    const nome = xml.split("<nome>")[1].split("</nome>")[0];
    return JSON.stringify({ nome });
  }
}
function exibir(fonte: FonteJson) {
  console.log(fonte.obterJson());
}
exibir(new AdaptadorXmlParaJson(new SistemaLegadoXml())); //
{"nome":"Ana"}
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se quer usar uma classe existente, mas a sua interface não é compatível com o resto do código, e quando se quer reutilizar várias subclasses que não têm alguma funcionalidade em comum, que não pode ser adicionada à superclasse.

Vantagens e desvantagens. A lógica de conversão fica separada da lógica de negócio (SRP) e novos adaptadores podem ser introduzidos sem quebrar o código cliente (OCP). Por outro lado, a complexidade geral aumenta, porque são adicionadas novas interfaces e classes, e às vezes é mais simples alterar diretamente a classe que fornece o serviço.

Relação com outros padrões. O Bridge é projetado antes, para que abstração e implementação variem de forma independente, enquanto o Adapter costuma ser usado depois, para fazer classes que já existem trabalharem juntas. O Adapter muda a interface de um objeto existente, o Decorator melhora o objeto sem mudar a interface, e o Proxy mantém a mesma interface.

3.2 Bridge

Também conhecido como: Ponte.

Intenção. Dividir uma classe grande, ou um conjunto de classes muito relacionadas, em duas hierarquias separadas, a abstração e a implementação, que podem ser desenvolvidas de forma independente.

Problema. Suponha uma classe Forma com as subclasses Círculo e Quadrado. Se for preciso adicionar cores, como vermelho e azul, será necessário criar quatro classes (Círculo Vermelho, Círculo Azul, Quadrado Vermelho e Quadrado Azul), e esse número cresce de forma exponencial a cada forma ou cor nova. O problema acontece porque a classe está tentando crescer em duas dimensões ao mesmo tempo (forma e cor) usando só herança.

Solução. O padrão troca a herança pela composição. Uma das dimensões é extraída para uma hierarquia própria, e a classe original passa a ter uma referência a um objeto dessa nova hierarquia, em vez de ter todo o comportamento dentro dela. No exemplo, a cor vira uma hierarquia separada, e a classe Forma guarda uma referência a um objeto Cor. Assim, as duas dimensões variam de forma independente.

Estrutura.

Abstração: fornece a lógica de controle de alto nível e delega o trabalho real ao objeto de implementação.
Implementação: interface comum a todas as implementações concretas.
Implementações concretas: contêm o código específico de cada plataforma ou variação.
Abstrações refinadas: variantes da lógica de controle, que trabalham com a mesma interface da implementação.
Cliente: normalmente só se interessa pela abstração.

Como implementar. (1) Identificar as dimensões independentes da classe (por exemplo, abstração e plataforma). (2) Definir as operações que o cliente precisa e declará-las na abstração base. (3) Declarar as operações disponíveis em todas as plataformas na interface da implementação. (4) Criar as implementações concretas. (5) Adicionar na abstração um campo que guarda a implementação e delegar a ela o trabalho.

Exemplo de código. O Código 7 usa controles remotos (abstração) que operam dispositivos diferentes (implementação).

Código 7 - Exemplo de Bridge em TypeScript

```
interface Dispositivo {
  ligar(): string;
  desligar(): string;
}
class Tv implements Dispositivo {
  ligar() { return "TV ligada"; }
  desligar() { return "TV desligada"; }
}
class Radio implements Dispositivo {
  ligar() { return "Rádio ligado"; }
  desligar() { return "Rádio desligado"; }
}
class ControleRemoto {
  constructor(protected dispositivo: Dispositivo) {}
  apertarPower() { return this.dispositivo.ligar(); }
}
class ControleAvancado extends ControleRemoto {
  desligarTudo() { return this.dispositivo.desligar(); }
}
const controle = new ControleAvancado(new Radio());
console.log(controle.apertarPower()); // Rádio ligado
console.log(controle.desligarTudo()); // Rádio desligado
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se quer dividir uma classe monolítica que tem várias variantes de alguma funcionalidade (por exemplo, trabalhar com vários servidores de banco de dados), quando é preciso estender uma classe em várias dimensões independentes e quando se quer poder trocar a implementação em tempo de execução.

Vantagens e desvantagens. Permite criar aplicações independentes de plataforma, faz o código cliente trabalhar com abstrações de alto nível e respeita o SRP e o OCP. A desvantagem é que o código pode ficar mais complicado se o padrão for aplicado a uma classe que já é bem coesa.

Relação com outros padrões. O Bridge costuma ser projetado desde o início, e o Adapter é usado com um sistema já existente. O Bridge, o State, o Strategy e, de certa forma, o Adapter têm estruturas parecidas, baseadas em composição e delegação, mas resolvem problemas diferentes. Uma Abstract Factory pode trabalhar junto com o Bridge quando algumas abstrações só funcionam com certas implementações.

3.3 Composite

Também conhecido como: Objeto Composto, Object Tree.

Intenção. Permitir compor objetos em estruturas de árvore e trabalhar com essas estruturas como se fossem objetos individuais.

Problema. O padrão só faz sentido quando o modelo central da aplicação pode ser representado como uma árvore. Imagine um pedido que contém produtos e caixas, e cada caixa pode ter outros produtos e até outras caixas menores. Para calcular o preço total, seria preciso abrir cada caixa, percorrer os itens e somar tudo, sabendo de antemão o tipo de cada elemento e o nível de aninhamento. Fazer isso diretamente no código é complicado ou até inviável.

Solução. Cria-se uma interface comum para produtos e caixas, com um método que calcula o preço. No caso de um produto, o método simplesmente retorna o valor. No caso de uma caixa, o método percorre os seus itens, pede o preço de cada um (que pode ser outra caixa) e soma os resultados. Assim, o cliente trata todos os elementos da mesma forma, sem precisar saber qual é a classe concreta de cada um. Sistemas de arquivos, com pastas e arquivos, e componentes de interface gráfica são exemplos conhecidos.

Estrutura.

Componente: interface que descreve as operações comuns à parte simples e à parte complexa da árvore.
Folha: elemento básico, que não tem subelementos e faz o trabalho de fato.
Contêiner (ou composto): elemento que tem subelementos, que podem ser folhas ou outros contêineres. Ele delega o trabalho para os filhos e soma os resultados.
Cliente: trabalha com todos os elementos pela interface do componente.

Como implementar. (1) Garantir que o modelo da aplicação pode ser representado como uma árvore. (2) Declarar a interface do componente com os métodos que fazem sentido para folhas e contêineres. (3) Criar a classe folha. (4) Criar a classe contêiner, com uma lista de filhos e métodos para adicionar e remover elementos. (5) Implementar os métodos do componente no contêiner, delegando o trabalho aos filhos.

Exemplo de código. O Código 8 calcula o preço de um pedido que tem produtos e caixas aninhadas.

Código 8 - Exemplo de Composite em TypeScript

```
interface Item {
  preco(): number;
}
class Produto implements Item {
  constructor(private valor: number) {}
  preco() { return this.valor; }
}
class Caixa implements Item {
  private itens: Item[] = [];
  adicionar(item: Item) { this.itens.push(item); }
  preco() {
    return this.itens.reduce((total, item) => total + item.preco(), 0);
  }
}
const caixaPequena = new Caixa();
caixaPequena.adicionar(new Produto(20));
const pedido = new Caixa();
pedido.adicionar(new Produto(50));
pedido.adicionar(caixaPequena);
console.log(pedido.preco()); // 70
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se precisa implementar uma estrutura de objetos em forma de árvore, e quando se quer que o código cliente trate elementos simples e complexos da mesma maneira.

Vantagens e desvantagens. Facilita o trabalho com estruturas complexas, usando polimorfismo e recursão, e respeita o OCP, pois novos tipos de elemento podem ser introduzidos sem quebrar o código existente. A desvantagem é que pode ser difícil criar uma interface comum para classes cujas funcionalidades são muito diferentes, e, em certos casos, seria preciso generalizar demais a interface, deixando o projeto difícil de entender.

Relação com outros padrões. O Builder pode ser usado para montar árvores Composite, e o Chain of Responsibility costuma ser usado junto com ele. É possível percorrer a árvore com o Iterator e aplicar uma operação a ela inteira com o Visitor. Os nós folha compartilhados podem ser implementados como Flyweights para economizar memória. O Decorator tem uma estrutura parecida, mas ele só tem um filho e adiciona responsabilidades, enquanto o Composite soma os resultados dos filhos.

3.4 Decorator

Também conhecido como: Decorador, Wrapper.

Intenção. Permitir que novos comportamentos sejam adicionados a objetos, colocando-os dentro de objetos especiais que contêm esses comportamentos.

Problema. Suponha uma biblioteca de notificações que, no começo, só envia e-mails. Com o tempo, alguns usuários querem receber também por SMS, outros pelo Slack, e outros por todos os canais. Resolver isso com herança cria uma explosão de subclasses para cada combinação possível. Além disso, a herança é estática (não dá para mudar o comportamento de um objeto em tempo de execução) e, na maioria das linguagens, uma classe só pode ter uma superclasse.

Solução. O padrão usa agregação ou composição em vez de herança. Um objeto (o decorador) tem uma referência a outro objeto, e os dois seguem a mesma interface. O decorador delega o trabalho ao objeto envolvido e acrescenta o seu comportamento antes ou depois. Como todos compartilham a mesma interface, é possível empilhar vários decoradores, um dentro do outro, combinando os comportamentos. Os fluxos de entrada e saída (streams) de Java são um exemplo conhecido.

Estrutura.

Componente: interface comum aos objetos simples e aos decoradores.
Componente concreto: a classe dos objetos que serão envolvidos, com o comportamento básico.
Decorador base: tem um campo para a referência ao objeto envolvido e delega todo o trabalho a ele.
Decoradores concretos: acrescentam comportamentos extras antes ou depois de chamar o método do objeto envolvido.
Cliente: pode envolver os componentes em várias camadas de decoradores.

Como implementar. (1) Identificar o comportamento básico e as funcionalidades opcionais. (2) Criar a interface do componente. (3) Criar a classe do componente concreto. (4) Criar o decorador base, com o campo da referência ao objeto envolvido. (5) Criar os decoradores concretos, estendendo o decorador base. (6) Montar as combinações no código cliente.

Exemplo de código. O Código 9 empilha notificações por e-mail, SMS e Slack.

Código 9 - Exemplo de Decorator em TypeScript

```
interface Notificador {
  enviar(msg: string): string;
}
class NotificadorEmail implements Notificador {
  enviar(msg: string) { return `E-mail: ${msg}`; }
}
class DecoradorBase implements Notificador {
  constructor(protected envolvido: Notificador) {}
  enviar(msg: string) { return this.envolvido.enviar(msg); }
}
class DecoradorSms extends DecoradorBase {
  enviar(msg: string) { return super.enviar(msg) + ` | SMS: ${msg}`; }
}
class DecoradorSlack extends DecoradorBase {
  enviar(msg: string) { return super.enviar(msg) + ` | Slack: ${msg}`; }
}
const notificador = new DecoradorSlack(new DecoradorSms(new
NotificadorEmail()));
console.log(notificador.enviar("Servidor fora do ar"));
// E-mail: Servidor fora do ar | SMS: Servidor fora do ar | Slack:
Servidor fora do ar
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando é preciso atribuir comportamentos extras a objetos em tempo de execução sem quebrar o código que os usa, e quando não dá para estender o comportamento usando herança, por exemplo em classes finais.

Vantagens e desvantagens. É possível estender o comportamento de um objeto sem criar uma subclasse, adicionar ou remover responsabilidades em tempo de execução, combinar vários comportamentos e dividir uma classe grande em classes menores (SRP). As desvantagens são a dificuldade de remover um decorador específico do meio da pilha, a dependência da ordem em que eles são empilhados e o código de configuração inicial, que pode ficar feio.

Relação com outros padrões. O Adapter muda a interface de um objeto, o Decorator melhora o objeto sem mudar a interface e o Proxy mantém a mesma interface, mas controla o acesso. O Composite e o Decorator têm estruturas parecidas, mas objetivos diferentes. Também é parecido com o Chain of Responsibility, mas, no Decorator, as camadas não interrompem o fluxo. O Strategy muda o interior do objeto, enquanto o Decorator muda a parte externa.

3.5 Facade

Também conhecido como: Fachada.

Intenção. Fornecer uma interface simplificada para uma biblioteca, um framework ou qualquer conjunto complexo de classes.

Problema. Para usar uma biblioteca ou um subsistema complexo, o código precisa inicializar dezenas de objetos, acompanhar as dependências e executar os métodos na ordem certa. Com isso, a lógica de negócio fica fortemente acoplada aos detalhes dessas classes, e fica difícil de entender e manter.

Solução. A fachada é uma classe que oferece uma interface simples para o subsistema. Ela pode ter menos funcionalidades do que se usar o subsistema diretamente, mas inclui somente o que o cliente realmente precisa. É como ligar para uma loja por telefone: o atendente funciona como a fachada de todo o processo interno de pedido, pagamento e entrega.

Estrutura.

Fachada: dá acesso prático a uma parte do subsistema e sabe para onde direcionar cada chamada.
Fachada adicional (opcional): evita que uma única fachada vire grande demais.
Subsistema: formado por dezenas de objetos que fazem o trabalho de fato e que não sabem da existência da fachada.
Cliente: usa a fachada em vez de chamar os objetos do subsistema diretamente.

Como implementar. (1) Verificar se é possível oferecer uma interface mais simples do que a que o subsistema já tem. (2) Declarar e implementar essa interface em uma nova classe de fachada, redirecionando as chamadas para os objetos certos. (3) Fazer o código cliente usar apenas a fachada. (4) Se a fachada ficar grande demais, dividi-la em fachadas menores.

Exemplo de código. O Código 10 esconde as etapas de conversão de um vídeo atrás de um único método.

Código 10 - Exemplo de Facade em TypeScript

```
class Codec {
  extrair(arquivo: string) { return `streams de ${arquivo} extraídos`; }
}
class MisturadorDeAudio {
  misturar() { return "áudio ajustado"; }
}
class Exportador {
  exportar(formato: string) { return `arquivo exportado em ${formato}`;
}
}
class FachadaConversor {
  converter(arquivo: string, formato: string): string {
    return [
      new Codec().extrair(arquivo),
      new MisturadorDeAudio().misturar(),
      new Exportador().exportar(formato),
    ].join(" > ");
  }
}
console.log(new FachadaConversor().converter("aula.avi", "mp4"));
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se precisa de uma interface limitada, mas simples, para um subsistema complexo, e quando se quer dividir o subsistema em camadas, usando fachadas como ponto de entrada de cada nível.

Vantagens e desvantagens. O código cliente fica isolado da complexidade do subsistema e o acoplamento diminui. A desvantagem é que a fachada pode se tornar um "objeto deus" (God Object), acoplado a todas as classes da aplicação e concentrando responsabilidades demais.

Relação com outros padrões. O Facade define uma nova interface para objetos existentes, enquanto o Adapter tenta fazer a interface existente ser utilizável. O Abstract Factory pode ser uma alternativa quando só se quer esconder a criação dos objetos do subsistema. O Mediator é parecido, pois organiza a comunicação entre classes, mas ele centraliza a comunicação entre os colegas e eles dependem do mediador, já na fachada o subsistema não sabe que ela existe. Uma fachada costuma poder ser um Singleton.

3.6 Flyweight

Também conhecido como: Peso Mosca, Cache.

Intenção. Permitir que mais objetos caibam na memória disponível, compartilhando as partes comuns do estado entre vários objetos, em vez de manter todos os dados em cada um deles.

Problema. Imagine um jogo com milhões de partículas, como balas, mísseis e fumaça. Cada partícula é um objeto com dados como cor, imagem (sprite), coordenadas e velocidade. Os campos de cor e imagem ocupam bastante memória e são iguais em muitas partículas, mas se repetem em cada objeto. No fim, o jogo consome toda a memória do computador e trava.

Solução. O padrão divide o estado dos objetos em duas partes. O estado intrínseco é o que não muda e se repete entre vários objetos (como a cor e a imagem), e fica guardado em um objeto flyweight, que é compartilhado. O estado extrínseco é o que é único de cada objeto (como as coordenadas e a velocidade) e é guardado no contexto ou passado por parâmetro aos métodos. Os objetos flyweight devem ser imutáveis, e uma fábrica de flyweights costuma gerenciar o pool de objetos já criados.

Estrutura.

Flyweight: contém a parte do estado que pode ser compartilhada por vários objetos.
Contexto: contém o estado extrínseco, único de cada objeto, e uma referência ao flyweight.
Fábrica de flyweights: gerencia os objetos já criados e devolve um existente ou cria um novo quando precisa.
Cliente: calcula ou guarda o estado extrínseco dos flyweights.

Como implementar. (1) Dividir os campos da classe em estado intrínseco e extrínseco. (2) Deixar os campos intrínsecos imutáveis, passando os valores só pelo construtor. (3) Mover o estado extrínseco para os parâmetros dos métodos ou para o contexto. (4) Criar uma fábrica que reaproveita os flyweights. (5) Fazer o cliente obter os flyweights pela fábrica.

Exemplo de código. O Código 11 compartilha o tipo da partícula (cor e imagem) entre todas as partículas parecidas.

Código 11 - Exemplo de Flyweight em TypeScript

```
class TipoParticula {
  constructor(readonly cor: string, readonly sprite: string) {}
  desenhar(x: number, y: number) {
    return `${this.sprite} ${this.cor} em (${x}, ${y})`;
  }
}
class FabricaDeTipos {
  private static tipos = new Map<string, TipoParticula>();
  static obter(cor: string, sprite: string): TipoParticula {
    const chave = `${cor}-${sprite}`;
    if (!this.tipos.has(chave)) {
      this.tipos.set(chave, new TipoParticula(cor, sprite));
    }
    return this.tipos.get(chave)!;
  }
}
class Particula {
  constructor(private x: number, private y: number, readonly tipo:
TipoParticula) {}
  desenhar() { return this.tipo.desenhar(this.x, this.y); }
}
const p1 = new Particula(1, 2, FabricaDeTipos.obter("vermelho",
"bala"));
const p2 = new Particula(5, 8, FabricaDeTipos.obter("vermelho",
"bala"));
console.log(p1.tipo === p2.tipo); // true: o mesmo objeto é
compartilhado
```

Fonte: elaborado pelo autor (2026).

Quando usar. Só quando o programa precisa criar uma quantidade enorme de objetos parecidos, que esgotam a memória disponível, e quando esses objetos têm estado duplicado que pode ser extraído e compartilhado.

Vantagens e desvantagens. A principal vantagem é a grande economia de memória quando existem muitos objetos semelhantes. Como desvantagens, pode haver gasto de processamento para recalcular o contexto sempre que alguém chama um método de flyweight, e o código fica mais complicado, já que a divisão do estado pode confundir quem não conhece o padrão.

Relação com outros padrões. Os nós folha de uma árvore Composite podem ser implementados como Flyweights para economizar memória. O Flyweight lembra o Singleton, mas pode ter várias instâncias com estados intrínsecos diferentes, e os seus objetos são imutáveis. Os estados do State e os objetos do Strategy também costumam ser Flyweights.

3.7 Proxy

Também conhecido como: Procurador, Substituto

Intenção. Fornecer um substituto, ou um espaço reservado, para outro objeto, controlando o acesso a ele e permitindo executar ações antes ou depois que a requisição chegue ao objeto original.

Problema. Imagine um objeto pesado, que consome muitos recursos do sistema e que só é necessário de vez em quando, como uma conexão com um banco de dados ou um serviço externo. Criar o objeto logo no início da aplicação seria um desperdício, e colocar o código de controle de acesso, cache ou registro (log) dentro do próprio objeto, ou em todos os clientes, duplicaria código e misturaria responsabilidades.

Solução. Cria-se uma classe proxy com a mesma interface do objeto de serviço real. O proxy recebe a requisição do cliente, faz o seu trabalho (como controlar o acesso, guardar em cache ou criar o objeto real apenas quando for realmente necessário) e então repassa o pedido ao objeto de serviço. Como o proxy tem a mesma interface, ele pode ser passado para qualquer cliente que espera o objeto real. Um cartão de crédito é um exemplo do dia a dia: ele é um proxy de uma conta bancária, que por sua vez é um proxy do dinheiro em espécie.

Estrutura. Existem vários tipos de proxy:

Proxy virtual: adia a criação de um objeto pesado até que ele seja realmente necessário.
Proxy de proteção: controla o acesso ao objeto conforme as permissões do cliente.
Proxy remoto: representa um objeto que está em outro servidor ou em outro processo.
Proxy de cache: guarda os resultados de requisições repetidas.
Proxy de registro (logging): grava o histórico das chamadas ao serviço.

Os participantes são a interface do serviço, o serviço (que tem a lógica de negócio de fato), o proxy (que guarda uma referência ao serviço) e o cliente (que trabalha com ambos pela mesma interface).

Como implementar. (1) Se não houver uma interface de serviço, criar uma para que proxy e serviço sejam intercambiáveis. (2) Criar a classe proxy com um campo que guarda a referência ao serviço real. (3) Implementar os métodos do proxy, adicionando o comportamento desejado e delegando o trabalho ao serviço. (4) Decidir se o serviço é criado pelo cliente ou pelo próprio proxy.

Exemplo de código. O Código 12 mostra um proxy de cache, que só consulta o serviço na primeira vez.

Código 12 - Exemplo de Proxy em TypeScript

```
interface Servico {
  buscar(id: number): string;
}
class ServicoReal implements Servico {
  buscar(id: number) { return `Dados do item ${id}`; }
}
class ProxyComCache implements Servico {
  private cache = new Map<number, string>();
  constructor(private real: Servico) {}

  buscar(id: number) {
    if (!this.cache.has(id)) {
      console.log("Buscando no serviço real...");
      this.cache.set(id, this.real.buscar(id));
    }
    return this.cache.get(id)!;
  }
}
const servico = new ProxyComCache(new ServicoReal());
servico.buscar(1); // Buscando no serviço real...
servico.buscar(1); // vem do cache, sem buscar de novo
```

Fonte: elaborado pelo autor (2026).

Quando usar. Na inicialização preguiçosa (proxy virtual), no controle de acesso (proxy de proteção), na execução local de um serviço remoto (proxy remoto), no registro de requisições (proxy de registro) e no armazenamento de resultados (proxy de cache).

Vantagens e desvantagens. É possível controlar o objeto de serviço sem que o cliente perceba, gerenciar o ciclo de vida do serviço quando o cliente não se importa com isso, e o proxy funciona mesmo que o serviço não esteja pronto. Também respeita o OCP, pois dá para criar novos proxies sem mudar o serviço ou os clientes. As desvantagens são o aumento do número de classes e a possível demora na resposta do serviço.

Relação com outros padrões. O Adapter fornece uma interface diferente para o objeto, o Proxy fornece a mesma interface e o Decorator fornece uma interface melhorada. O Decorator e o Proxy têm estruturas parecidas, mas o Decorator permite compor várias camadas dinamicamente, e o Proxy normalmente gerencia o ciclo de vida do serviço por conta própria. O Facade também é parecido, mas fornece uma interface simples para vários objetos, não só para um.

4 PADRÕES COMPORTAMENTAIS

Os padrões comportamentais se preocupam com os algoritmos e com a distribuição de responsabilidades entre os objetos (REFACTORING.GURU, 2026). Eles descrevem não só os padrões de objetos e classes, mas também a forma como os objetos se comunicam entre si.

4.1 Chain of Responsibility

Também conhecido como: Corrente de Responsabilidade, Cadeia de Comando, CoR, Chain of Command.

Intenção. Permitir passar pedidos por uma corrente de manipuladores (handlers). Ao receber um pedido, cada manipulador decide se processa o pedido ou se o passa adiante para o próximo da corrente.

Problema. Imagine um sistema que precisa fazer várias verificações em uma requisição antes de processá-la: autenticação, autorização, validação dos dados, cache e assim por diante. Com o tempo, novas verificações vão sendo adicionadas ao mesmo bloco de código, que cresce, se repete em vários lugares e fica difícil de manter e de reutilizar.

Solução. Cada verificação é transformada em uma classe independente, chamada de manipulador, com um único método de execução. Os manipuladores são ligados em uma corrente, em que cada um guarda uma referência ao próximo. Quando recebe o pedido, o manipulador pode tratá-lo e encerrar a corrente, ou passá-lo adiante. Os middlewares de frameworks web, como o Express, funcionam de uma forma bem parecida.

Estrutura.

Manipulador: interface comum a todos os manipuladores concretos, com o método para tratar o pedido e, às vezes, o método para definir o próximo.
Manipulador base (opcional): guarda a referência ao próximo manipulador e implementa o comportamento padrão de repassar o pedido.
Manipuladores concretos: contêm o código que processa o pedido e decidem se o repassam ou não.
Cliente: monta a corrente e envia o pedido ao primeiro manipulador.

Como implementar. (1) Declarar a interface do manipulador. (2) Criar a classe manipuladora base, com o campo do próximo manipulador. (3) Criar os manipuladores concretos, decidindo em cada um se trata o pedido ou o repassa. (4) Montar a corrente no cliente, na ordem desejada.

Exemplo de código. O Código 13 monta uma corrente com autenticação, autorização e validação.

Código 13 - Exemplo de Chain of Responsibility em TypeScript

```
interface Requisicao { usuario?: string; admin: boolean; dados: string;
}
abstract class Manipulador {
  private proximo?: Manipulador;
  definirProximo(m: Manipulador): Manipulador {
    this.proximo = m;
    return m;
  }
  tratar(req: Requisicao): string {
    return this.proximo ? this.proximo.tratar(req) : "Requisição
aprovada";
  }
}
class Autenticacao extends Manipulador {
  tratar(req: Requisicao) {
    return req.usuario ? super.tratar(req) : "Erro: usuário não
autenticado";
  }
}
class Autorizacao extends Manipulador {
  tratar(req: Requisicao) {
    return req.admin ? super.tratar(req) : "Erro: sem permissão";
  }
}
class Validacao extends Manipulador {
  tratar(req: Requisicao) {
    return req.dados !== "" ? super.tratar(req) : "Erro: dados vazios";
  }
}
const cadeia = new Autenticacao();
cadeia.definirProximo(new Autorizacao()).definirProximo(new
Validacao());
console.log(cadeia.tratar({ usuario: "ana", admin: false, dados: "x"
})); // Erro: sem permissão
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando o programa precisa processar diferentes tipos de requisições de várias maneiras, mas os tipos exatos e as sequências não são conhecidos de antemão; quando é essencial executar vários manipuladores em uma ordem específica; e quando o conjunto de manipuladores e a ordem deles precisam mudar em tempo de execução.

Vantagens e desvantagens. É possível controlar a ordem de tratamento dos pedidos, e o padrão respeita o SRP (as classes que invocam as operações ficam separadas das que as executam) e o OCP (novos manipuladores entram sem quebrar o código). Em compensação, alguns pedidos podem acabar sem ser tratados por nenhum manipulador.

Relação com outros padrões. O Chain of Responsibility, o Command, o Mediator e o Observer tratam de formas diferentes de conectar remetentes e destinatários. Costuma ser usado junto com o Composite, caso em que o pedido sobe pela árvore. Os manipuladores podem ser implementados como Commands. A estrutura é parecida com a do Decorator, mas no Chain of Responsibility os manipuladores podem interromper o fluxo.

4.2 Command

Também conhecido como: Comando, Ação (Action), Transação (Transaction).

Intenção. Transformar uma requisição em um objeto independente, que contém todas as informações sobre ela. Essa transformação permite passar requisições como argumentos de métodos, atrasar ou colocar a requisição em uma fila e dar suporte a operações que podem ser desfeitas.

Problema. Imagine um editor de texto com uma barra de ferramentas cheia de botões, e também menus e atalhos de teclado que executam as mesmas ações, como copiar e colar. Se for criada uma subclasse de botão para cada operação, o código de interface fica fortemente acoplado à lógica de negócio, e o código das operações acaba duplicado em vários lugares.

Solução. Uma boa arquitetura separa a interface gráfica da lógica de negócio. Nessa divisão, a interface não executa o trabalho diretamente, mas cria um comando e o dispara. O comando, por sua vez, delega o trabalho a um objeto receptor, que contém a lógica de fato. Uma analogia é o pedido feito em um restaurante: o garçom anota o pedido em um papel (o comando) e o entrega na cozinha, sem precisar saber como o prato é feito. Como o pedido virou um objeto, também é possível guardar um histórico dele e desfazê-lo.

Estrutura.

Invocador (sender): guarda a referência a um comando e o aciona, sem saber qual comando concreto é.
Comando: interface que costuma ter um único método de execução.
Comandos concretos: implementam vários tipos de requisições, delegando o trabalho a um receptor.
Receptor: contém a lógica de negócio de fato.
Cliente: cria e configura os comandos concretos.

Como implementar. (1) Declarar a interface do comando, com um único método de execução. (2) Extrair cada requisição para uma classe de comando concreto, guardando os parâmetros em campos. (3) Fazer os invocadores guardarem uma referência a um comando. (4) Fazer os invocadores executarem o comando, em vez de falarem diretamente com o receptor. (5) Montar o cliente, que cria os comandos e os associa aos invocadores.

Exemplo de código. O Código 14 implementa o botão de desfazer de um editor de texto.

Código 14 - Exemplo de Command em TypeScript

```
interface Comando {
  executar(): void;
  desfazer(): void;
}
class Editor {
  texto = "";
}
class ComandoEscrever implements Comando {
  constructor(private editor: Editor, private trecho: string) {}
  executar() { this.editor.texto += this.trecho; }
  desfazer() {
    this.editor.texto = this.editor.texto.slice(0, -this.trecho.length);
  }
}
class Historico {
  private pilha: Comando[] = [];
  executar(c: Comando) { c.executar(); this.pilha.push(c); }
  desfazer() { this.pilha.pop()?.desfazer(); }
}
const editor = new Editor();
const historico = new Historico();
historico.executar(new ComandoEscrever(editor, "Olá "));
historico.executar(new ComandoEscrever(editor, "mundo"));
console.log(editor.texto); // Olá mundo
historico.desfazer();
console.log(editor.texto); // Olá
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se quer parametrizar objetos com operações; quando se quer enfileirar operações, agendar a sua execução ou executá-las remotamente; e quando se quer implementar operações reversíveis, como desfazer e refazer.

Vantagens e desvantagens. O padrão desacopla as classes que invocam operações das que as executam, respeita o SRP e o OCP, permite implementar desfazer e refazer, adiar a execução e montar comandos complexos a partir de comandos simples. A desvantagem é que o código pode ficar mais complicado, por causa de uma nova camada entre remetentes e destinatários.

Relação com outros padrões. O Chain of Responsibility, o Command, o Mediator e o Observer tratam de maneiras diferentes de conectar remetentes e destinatários. Os manipuladores do Chain of Responsibility podem ser implementados como Commands. O Command e o Memento costumam ser usados juntos para implementar o desfazer: o comando executa a operação e o memento guarda o estado anterior. O Prototype pode ajudar a guardar cópias dos comandos no histórico. O Strategy parece com o Command, mas o Strategy descreve diferentes formas de fazer a mesma coisa, enquanto o Command transforma qualquer ação em objeto.

4.3 Iterator

Também conhecido como: Iterador.

Intenção. Permitir percorrer os elementos de uma coleção sem expor a sua representação interna, seja ela uma lista, uma pilha, uma árvore ou um grafo.

Problema. As coleções são um dos tipos de dados mais usados, mas cada uma guarda os elementos de um jeito diferente, e cada uma tem uma forma própria de ser percorrida. Quando a lógica de percurso fica dentro da coleção, ela se enche de código que não é a sua função principal, e o código cliente acaba dependendo da estrutura interna de cada tipo de coleção. Além disso, algumas estruturas, como árvores, podem ser percorridas de várias maneiras (em profundidade, em largura etc.).

Solução. O comportamento de percurso é extraído da coleção e colocado em um objeto separado, chamado de iterador. Ele encapsula os detalhes do percurso, como a posição atual e quantos elementos faltam, e costuma oferecer métodos para obter o próximo elemento e para verificar se ainda há elementos. Como todos os iteradores seguem a mesma interface, o cliente pode percorrer qualquer coleção da mesma maneira. Muitas linguagens já trazem o padrão embutido, como o for...of do JavaScript.

Estrutura.

Iterador: interface que declara as operações para percorrer a coleção (obter o próximo elemento, verificar se há mais, e assim por diante).
Iteradores concretos: implementam algoritmos específicos de percurso e acompanham o progresso por conta própria.
Coleção: interface que declara um ou mais métodos para obter iteradores compatíveis.
Coleções concretas: retornam novos iteradores de uma classe específica a cada pedido.
Cliente: trabalha com coleções e iteradores pelas interfaces.

Como implementar. (1) Declarar a interface do iterador, com pelo menos um método para buscar o próximo elemento. (2) Declarar a interface da coleção, com um método para criar iteradores. (3) Implementar as classes de iteradores concretos para as coleções que se quer percorrer. (4) Implementar a criação do iterador nas coleções concretas. (5) Trocar o código de percurso do cliente pelo uso do iterador.

Exemplo de código. O Código 15 percorre uma playlist sem expor como as músicas são guardadas.

Código 15 - Exemplo de Iterator em TypeScript

```
interface Iterador<T> {
  temProximo(): boolean;
  proximo(): T;
}
class IteradorLista<T> implements Iterador<T> {
  private posicao = 0;
  constructor(private itens: T[]) {}
  temProximo() { return this.posicao < this.itens.length; }
  proximo() { return this.itens[this.posicao++]; }
}
class Playlist {
  private musicas: string[] = [];
  adicionar(musica: string) { this.musicas.push(musica); }
  criarIterador(): Iterador<string> { return new
IteradorLista(this.musicas); }
}
const playlist = new Playlist();
playlist.adicionar("Música A");
playlist.adicionar("Música B");
const it = playlist.criarIterador();
while (it.temProximo()) {
  console.log(it.proximo()); // Música A, depois Música B
}
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando a coleção tem uma estrutura complexa por baixo, mas se quer esconder essa complexidade do cliente; para reduzir a duplicação do código de percurso na aplicação; e quando se quer que o código consiga percorrer diferentes estruturas de dados, ou até estruturas desconhecidas de antemão.

Vantagens e desvantagens. O padrão respeita o SRP, já que o código de percurso sai da coleção, e o OCP, pois novas coleções e iteradores podem ser adicionados sem quebrar o código. Também é possível percorrer a mesma coleção em paralelo e pausar o percurso, continuando depois. Como desvantagens, o padrão pode ser um exagero se a aplicação só trabalha com coleções simples, e usar um iterador pode ser menos eficiente do que percorrer diretamente os elementos de algumas coleções especializadas.

Relação com outros padrões. Os iteradores podem ser usados para percorrer árvores Composite. Podem ser combinados com o Factory Method para que as subclasses da coleção retornem diferentes tipos de iteradores. O Memento pode ser usado com o Iterator para guardar o estado atual da iteração e voltar a ele, e o Visitor pode ser usado junto para executar uma operação sobre cada elemento percorrido.

4.4 Mediator

Também conhecido como: Mediador, Intermediário, Controlador.

Intenção. Reduzir as dependências caóticas entre objetos, restringindo a comunicação direta entre eles e forçando-os a colaborar apenas por meio de um objeto mediador.

Problema. Imagine uma caixa de diálogo de cadastro com campos de texto, caixas de seleção e botões que se influenciam entre si. Ao marcar uma opção, outro campo aparece, e o botão de enviar só habilita se os dados forem válidos. Se cada elemento conhecer diretamente os outros, as classes ficam fortemente acopladas e não podem ser reutilizadas em outros formulários.

Solução. Os componentes deixam de se conhecer e passam a notificar o mediador sobre os eventos que acontecem neles. O mediador, que conhece todos os componentes, reage e coordena as ações. Assim, cada componente só depende do mediador. Uma analogia é a torre de controle de um aeroporto: os pilotos não conversam entre si para decidir quem pousa primeiro, e toda a comunicação passa pela torre.

Estrutura.

Componentes: classes que têm alguma lógica de negócio e guardam uma referência ao mediador, sem saber qual é a classe dele.
Interface do mediador: declara os métodos de comunicação com os componentes, geralmente um único método de notificação.
Mediador concreto: conhece todos os componentes que gerencia e, às vezes, até cuida do ciclo de vida deles.

Como implementar. (1) Identificar as classes fortemente acopladas que se beneficiariam de ficarem independentes. (2) Declarar a interface do mediador com o método de notificação. (3) Implementar o mediador concreto, com as referências aos componentes. (4) Fazer os componentes guardarem a referência ao mediador e notificá-lo em vez de chamarem uns aos outros. (5) Mover a lógica de reação aos eventos para o mediador.

Exemplo de código. O Código 16 mostra um formulário em que a caixa de seleção mostra um campo extra, sem que os dois componentes se conheçam.

Código 16 - Exemplo de Mediator em TypeScript

```
interface Mediador {
  notificar(origem: object, evento: string): void;
}
class CampoTexto {
  visivel = false;
}
class CaixaSelecao {
  marcada = false;
  constructor(private mediador: Mediador) {}
  alternar() {
    this.marcada = !this.marcada;
    this.mediador.notificar(this, "alternou");
  }
}
class FormularioCadastro implements Mediador {
  caixaEmpresa = new CaixaSelecao(this);
  campoEmpresa = new CampoTexto();

  notificar(origem: object, evento: string) {
    if (origem === this.caixaEmpresa && evento === "alternou") {
      this.campoEmpresa.visivel = this.caixaEmpresa.marcada;
    }
  }
}
const form = new FormularioCadastro();
form.caixaEmpresa.alternar();
console.log(form.campoEmpresa.visivel); // true
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando é difícil alterar algumas classes porque elas estão fortemente acopladas a outras; quando não dá para reutilizar um componente em outro programa porque ele depende de muitos outros; e quando se criam muitas subclasses de componentes só para reaproveitar um comportamento em contextos diferentes.

Vantagens e desvantagens. O padrão respeita o SRP, pois extrai a comunicação entre os componentes para um único lugar, e o OCP, já que novos mediadores podem ser introduzidos sem alterar os componentes. Também reduz o acoplamento e facilita a reutilização dos componentes. A desvantagem é que, com o tempo, o mediador pode virar um "objeto deus", concentrando lógica demais.

Relação com outros padrões. O Chain of Responsibility, o Command, o Mediator e o Observer tratam de formas diferentes de conectar remetentes e destinatários. O Mediator e o Observer são parecidos, e muitas vezes um mediador é implementado com o Observer, mas o foco é diferente: o Mediator elimina as dependências mútuas entre os componentes, enquanto o Observer estabelece conexões de mão única, em que alguns objetos observam outros. O Facade também é parecido, mas ele não oferece comunicação entre os objetos do subsistema.

4.5 Memento

Também conhecido como: Lembrança, Snapshot, Token.

Intenção. Permitir salvar e restaurar o estado anterior de um objeto sem revelar os detalhes da sua implementação, preservando o encapsulamento.

Problema. Imagine um editor de texto que precisa de uma função de desfazer. Para guardar um "instantâneo" do estado do editor, seria preciso copiar o valor de todos os seus campos, mas muitos deles são privados, e o código de fora não consegue acessá-los. Tornar os campos públicos quebraria o encapsulamento, e qualquer mudança na classe quebraria o código que salva e restaura o estado.

Solução. A criação do instantâneo é delegada ao próprio objeto, chamado de originador (Originator). Ele produz um objeto especial, o memento, que guarda o estado e cujo conteúdo só o originador consegue ler. Um terceiro objeto, o zelador (Caretaker), guarda os mementos, por exemplo em uma pilha, mas não pode alterar o conteúdo deles. Quando o usuário pede para desfazer, o zelador devolve o último memento ao originador, que restaura o seu estado.

Estrutura.

Originador: cria os mementos com o seu estado atual e restaura o estado a partir deles.
Memento: objeto que guarda o estado do originador, geralmente imutável.
Zelador: sabe quando e por que salvar e restaurar o estado do originador, guardando o histórico de mementos.

Como implementar. (1) Definir a classe que será o originador. (2) Criar a classe do memento, com campos que espelham o estado do originador. (3) Fazer o memento ser imutável, recebendo os dados uma única vez, pelo construtor. (4) Adicionar ao originador o método para criar mementos e o método para restaurar o estado a partir deles. (5) Criar o zelador, que guarda os mementos, por exemplo em uma pilha.

Exemplo de código. O Código 17 implementa um desfazer simples para um editor de texto. Em TypeScript, o isolamento do memento é apenas aproximado, já que a linguagem não tem classes aninhadas com acesso privado como outras linguagens.

Código 17 - Exemplo de Memento em TypeScript

```
class Memento {
  constructor(private readonly estado: string) {}
  obterEstado() { return this.estado; }
}
class Editor {
  texto = "";
  salvar(): Memento { return new Memento(this.texto); }
  restaurar(m: Memento) { this.texto = m.obterEstado(); }
}
class Zelador {
  private pilha: Memento[] = [];
  guardar(m: Memento) { this.pilha.push(m); }
  desfazer(): Memento | undefined { return this.pilha.pop(); }
}
const editor = new Editor();
const zelador = new Zelador();
editor.texto = "Primeira versão";
zelador.guardar(editor.salvar());
editor.texto = "Segunda versão (com erro)";
const anterior = zelador.desfazer();
if (anterior) editor.restaurar(anterior);
console.log(editor.texto); // Primeira versão
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se quer produzir instantâneos do estado de um objeto para poder restaurá-lo depois, e quando o acesso direto aos campos do objeto violaria o seu encapsulamento.

Vantagens e desvantagens. É possível produzir instantâneos sem violar o encapsulamento e simplificar o código do originador, deixando o histórico a cargo do zelador. As desvantagens são o alto consumo de memória se os mementos forem criados com muita frequência, a necessidade de o zelador acompanhar o ciclo de vida do originador para descartar mementos obsoletos e a dificuldade de garantir que o estado guardado não seja alterado em linguagens dinâmicas, como PHP, Python e JavaScript.

Relação com outros padrões. O Command e o Memento costumam ser usados juntos para implementar o desfazer: o comando executa a operação e o memento guarda o estado anterior. O Memento pode ser usado com o Iterator para guardar o estado de uma iteração e voltar a ele. O Prototype pode ser uma alternativa mais simples ao Memento, quando o objeto cujo estado se quer guardar é simples e não tem ligações com recursos externos.

4.6 Observer

Também conhecido como: Observador, Publicador-Assinante (Publish-Subscribe), Event-Subscriber, Listener.

Intenção. Definir um mecanismo de assinatura (subscription) para notificar vários objetos sobre eventos que acontecem em outro objeto que eles estão observando.

Problema. Imagine dois tipos de objeto: o cliente e a loja. O cliente está interessado em um produto que ainda não chegou à loja. Se ele ficar visitando a loja todo dia, perde tempo à toa. Se a loja enviar e-mail sobre todo produto novo a todos os clientes, vai incomodar quem não tem interesse. Ou seja, ou o cliente gasta tempo verificando, ou a loja gasta recursos avisando as pessoas erradas.

Solução. O objeto que tem um estado de interesse é chamado de publicador (publisher) e mantém uma lista de assinantes (subscribers). Quando algo importante acontece, o publicador percorre essa lista e chama um método de notificação definido em uma interface comum. Os assinantes podem entrar e sair da lista a qualquer momento. Esse padrão é a base de muitos sistemas de eventos, incluindo os listeners de interfaces gráficas.

Estrutura.

Publicador: envia eventos de interesse a outros objetos e tem a infraestrutura para adicionar e remover assinantes.
Interface do assinante: declara o método de notificação.
Assinantes concretos: realizam ações em resposta às notificações do publicador.
Cliente: cria os publicadores e os assinantes e registra os assinantes.

Como implementar. (1) Separar o código em duas partes: a lógica principal, que será o publicador, e o restante, que serão os assinantes. (2) Declarar a interface do assinante com o método de notificação. (3) Declarar a interface do publicador, com métodos para adicionar e remover assinantes. (4) Guardar a lista de assinantes no publicador. (5) Chamar o método de notificação de cada assinante sempre que o evento ocorrer.

Exemplo de código. O Código 18 avisa os clientes quando um produto chega à loja.

Código 18 - Exemplo de Observer em TypeScript

```
interface Assinante {
  atualizar(produto: string): void;
}
class Loja {
  private assinantes: Assinante[] = [];
  assinar(a: Assinante) { this.assinantes.push(a); }
  cancelar(a: Assinante) {
    this.assinantes = this.assinantes.filter((x) => x !== a);
  }
  chegouProduto(produto: string) {
    this.assinantes.forEach((a) => a.atualizar(produto));
  }
}
class Cliente implements Assinante {
  constructor(private nome: string) {}
  atualizar(produto: string) {
    console.log(`${this.nome}, chegou: ${produto}`);
  }
}
const loja = new Loja();
loja.assinar(new Cliente("Ana"));
loja.assinar(new Cliente("Bruno"));
loja.chegouProduto("Teclado");
// Ana, chegou: Teclado
// Bruno, chegou: Teclado
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando a mudança no estado de um objeto pode exigir mudanças em outros objetos, e o conjunto desses objetos não é conhecido de antemão ou muda de forma dinâmica; e quando alguns objetos da aplicação precisam observar outros, mas só por um tempo ou em casos específicos.

Vantagens e desvantagens. O padrão respeita o OCP, pois novos assinantes podem ser adicionados sem alterar o publicador, e permite estabelecer relações entre objetos em tempo de execução. As desvantagens são que os assinantes são notificados em ordem aleatória e que o excesso de notificações pode causar efeitos colaterais difíceis de rastrear.

Relação com outros padrões. O Chain of Responsibility, o Command, o Mediator e o Observer tratam de formas diferentes de conectar remetentes e destinatários. A diferença do Observer para o Mediator é que o Mediator elimina as dependências mútuas entre os componentes, enquanto o Observer cria conexões dinâmicas de mão única. O Mediator pode ser implementado com o Observer.

4.7 State

Também conhecido como: Estado.

Intenção. Permitir que um objeto altere o seu comportamento quando o seu estado interno muda, de modo que pareça que o objeto mudou de classe.

Problema. O padrão está bastante relacionado ao conceito de máquina de estados finitos: um programa pode estar, a cada momento, em um número finito de estados e se comporta de forma diferente em cada um deles, passando de um estado para outro por meio de transições. Quando essa lógica é implementada com condicionais (if e switch) em todos os métodos, o código fica enorme e frágil, e adicionar um novo estado ou transição obriga a alterar vários lugares.

Solução. Cria-se uma classe para cada estado possível e o comportamento específico de cada estado é movido para a sua classe. O objeto original, chamado de contexto, guarda uma referência a um dos objetos de estado e delega a ele o trabalho relacionado ao estado. Para mudar o estado do contexto, basta trocar o objeto de estado ativo por outro, desde que todos sigam a mesma interface. Um exemplo é um documento que pode estar em rascunho, em moderação ou publicado, e cujo botão de publicar se comporta de um jeito diferente em cada situação.

Estrutura.

Contexto: guarda uma referência a um dos estados concretos e delega a ele o trabalho específico do estado.
Interface do estado: declara os métodos específicos de cada estado.
Estados concretos: implementam os métodos para o seu estado e podem guardar uma referência ao contexto para provocar a transição.

Como implementar. (1) Decidir qual classe será o contexto. (2) Declarar a interface do estado, com os métodos que dependem do estado. (3) Criar uma classe para cada estado concreto. (4) No contexto, adicionar um campo que guarda o estado atual e um método para trocá-lo. (5) Substituir as condicionais do contexto por chamadas ao estado atual. (6) Fazer os estados provocarem as transições quando for necessário.

Exemplo de código. O Código 19 mostra as transições de um documento entre rascunho, moderação e publicado.

Código 19 - Exemplo de State em TypeScript

```
interface EstadoDocumento {
  publicar(doc: Documento): void;
}
class Rascunho implements EstadoDocumento {
  publicar(doc: Documento) {
    console.log("Enviado para moderação");
    doc.mudarEstado(new Moderacao());
  }
}
class Moderacao implements EstadoDocumento {
  publicar(doc: Documento) {
    console.log("Documento publicado");
    doc.mudarEstado(new Publicado());
  }
}
class Publicado implements EstadoDocumento {
  publicar(doc: Documento) {
    console.log("O documento já está publicado");
  }
}
class Documento {
  private estado: EstadoDocumento = new Rascunho();
  mudarEstado(novo: EstadoDocumento) { this.estado = novo; }
  publicar() { this.estado.publicar(this); }
}
const doc = new Documento();
doc.publicar(); // Enviado para moderação
doc.publicar(); // Documento publicado
doc.publicar(); // O documento já está publicado
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando um objeto se comporta de maneiras diferentes dependendo do estado atual, quando o número de estados é grande e o código específico de cada estado muda com frequência, e quando uma classe está poluída com condicionais enormes que alteram o comportamento de acordo com os valores dos campos.

Vantagens e desvantagens. O código relacionado a cada estado fica organizado em classes separadas (SRP), novos estados podem ser introduzidos sem alterar os existentes (OCP) e as condicionais volumosas desaparecem. A desvantagem é que o padrão pode ser um exagero se a máquina de estados tem poucos estados ou raramente muda.

Relação com outros padrões. O State pode ser considerado uma extensão do Strategy, já que os dois são baseados em composição e delegação. A diferença é que, no Strategy, as estratégias são independentes e não se conhecem, enquanto no State os próprios estados podem provocar as transições para outros estados. O Bridge e o Adapter também têm estruturas parecidas, mas resolvem outros problemas.

4.8 Strategy

Também conhecido como: Estratégia.

Intenção. Definir uma família de algoritmos, colocar cada um deles em uma classe separada e tornar os seus objetos intercambiáveis.

Problema. Imagine um aplicativo de navegação que calcula rotas para carro, a pé, de bicicleta e de transporte público. Se todos os algoritmos de rota estiverem dentro da mesma classe, ela cresce a cada nova opção, fica difícil de manter, e qualquer alteração em um algoritmo pode causar erros nos outros. Além disso, o trabalho em equipe fica complicado, porque várias pessoas mexem na mesma classe.

Solução. Cada algoritmo é extraído para uma classe separada, chamada de estratégia, todas seguindo a mesma interface. A classe original, chamada de contexto, guarda uma referência a uma estratégia e delega o trabalho a ela, sem saber qual algoritmo concreto está sendo usado. O cliente escolhe a estratégia e a passa ao contexto, podendo trocá-la em tempo de execução.

Estrutura.

Contexto: guarda uma referência a uma estratégia e se comunica com ela somente pela interface.
Estratégia: interface comum a todas as estratégias concretas, que declara o método de execução do algoritmo.
Estratégias concretas: implementam variações do algoritmo.
Cliente: cria o objeto de estratégia desejado e o passa ao contexto.

Como implementar. (1) Identificar a classe que tem um algoritmo que muda com frequência ou que tem muitas condicionais para escolher uma variação. (2) Declarar a interface da estratégia. (3) Extrair cada variação do algoritmo para uma classe própria. (4) No contexto, adicionar um campo para a estratégia e um método para trocá-la. (5) Fazer o cliente associar o contexto à estratégia adequada.

Exemplo de código. O Código 20 calcula uma rota usando estratégias diferentes de deslocamento.

Código 20 - Exemplo de Strategy em TypeScript

```
interface EstrategiaRota {
  calcular(origem: string, destino: string): string;
}
class RotaCarro implements EstrategiaRota {
  calcular(origem: string, destino: string) {
    return `De carro, pela via mais rápida: ${origem} até ${destino}`;
  }
}
class RotaAPe implements EstrategiaRota {
  calcular(origem: string, destino: string) {
    return `A pé, pelo caminho mais curto: ${origem} até ${destino}`;
  }
}
class Navegador {
  constructor(private estrategia: EstrategiaRota) {}
  definirEstrategia(e: EstrategiaRota) { this.estrategia = e; }
  rota(origem: string, destino: string) {
    return this.estrategia.calcular(origem, destino);
  }
}
const nav = new Navegador(new RotaCarro());
console.log(nav.rota("Asa Sul", "Taguatinga"));
nav.definirEstrategia(new RotaAPe());
console.log(nav.rota("Asa Sul", "Rodoviária"));
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se quer usar diferentes variantes de um algoritmo dentro de um objeto e poder trocar de algoritmo em tempo de execução; quando há muitas classes parecidas que só diferem na forma como executam um comportamento; e quando é preciso isolar a lógica de negócio dos detalhes de implementação de um algoritmo.

Vantagens e desvantagens. É possível trocar os algoritmos usados dentro de um objeto em tempo de execução, isolar os detalhes de implementação do código que os usa, substituir a herança por composição e respeitar o OCP. As desvantagens são que o padrão é desnecessário se há poucos algoritmos que quase não mudam, e que o cliente precisa conhecer as diferenças entre as estratégias para escolher a adequada. Em linguagens modernas, funções anônimas também podem fazer esse papel sem criar classes novas.

Relação com outros padrões. O Bridge, o State, o Strategy e, de certa forma, o Adapter têm estruturas parecidas, baseadas em composição, mas resolvem problemas diferentes. O Command e o Strategy podem parecer iguais, mas o Command converte qualquer operação em objeto, enquanto o Strategy descreve diferentes formas de fazer a mesma coisa. O Decorator muda a aparência externa do objeto, e o Strategy muda o seu interior. O Template Method usa herança e funciona em nível de classe, enquanto o Strategy usa composição e funciona em nível de objeto.

4.9 Template Method

Também conhecido como: Método Modelo, Método Template.

Intenção. Definir o esqueleto de um algoritmo na superclasse, mas permitir que as subclasses sobrescrevam etapas específicas do algoritmo sem alterar a sua estrutura geral.

Problema. Imagine uma aplicação de mineração de dados que processa documentos em formatos diferentes, como DOC, CSV e PDF. O código de abrir o arquivo e extrair os dados é diferente para cada formato, mas as etapas de análise e de geração de relatórios são praticamente iguais. Sem o padrão, esse código fica repetido em cada classe, e qualquer mudança no algoritmo geral obriga a alterar todas elas. Além disso, o código cliente acaba cheio de condicionais para escolher a classe certa.

Solução. O algoritmo é dividido em passos, cada passo é transformado em um método e a sequência de chamadas é colocada em um único método, o método template. Os passos comuns ficam implementados na superclasse, e os passos que variam são abstratos ou sobrescritos nas subclasses. Também podem existir ganchos (hooks), que são passos opcionais com implementação vazia. O padrão se relaciona com o chamado "princípio de Hollywood": "não nos chame, nós chamamos você", já que a superclasse é quem chama as subclasses.

Estrutura.

Classe abstrata: declara os métodos que representam as etapas do algoritmo e o método template, que chama essas etapas em uma ordem específica.
Classes concretas: sobrescrevem todas as etapas abstratas e, se quiserem, também as opcionais, mas não o método template.

Como implementar. (1) Analisar o algoritmo e dividi-lo em etapas. (2) Criar a classe base abstrata, com o método template e as etapas como métodos. (3) Decidir quais etapas são abstratas, quais têm implementação padrão e quais são ganchos. (4) Criar uma subclasse concreta para cada variação do algoritmo. (5) Implementar nelas as etapas que faltam.

Exemplo de código. O Código 21 mostra a mineração de dados de um arquivo CSV.

Código 21 - Exemplo de Template Method em TypeScript

```
abstract class MineradorDeDados {
  // método template: define a ordem das etapas
  minerar(caminho: string): string {
    const bruto = this.abrirArquivo(caminho);
    const dados = this.extrairDados(bruto);
    const resultado = this.analisar(dados);
    this.enviarRelatorio(resultado);
    return resultado;
  }
  protected abstract abrirArquivo(caminho: string): string;
  protected abstract extrairDados(bruto: string): string[];
  protected analisar(dados: string[]): string {
    return `${dados.length} registros analisados`;
  }
  protected enviarRelatorio(resultado: string): void {
    console.log(`Relatório: ${resultado}`);
  }
}
class MineradorCsv extends MineradorDeDados {
  protected abrirArquivo(caminho: string) { return `conteúdo de
${caminho}`; }
  protected extrairDados(bruto: string) { return ["a", "b", "c"]; }
}
new MineradorCsv().minerar("vendas.csv"); // Relatório: 3 registros
analisados
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando se quer que os clientes estendam apenas etapas específicas de um algoritmo, mas não toda a sua estrutura, e quando várias classes têm algoritmos quase idênticos, com pequenas diferenças, de modo que alterar um algoritmo obriga a alterar todas as classes.

Vantagens e desvantagens. O padrão permite que os clientes sobrescrevam apenas certas partes de um algoritmo grande, afetando menos o resto, e reduz a duplicação de código, já que o que é comum vai para a superclasse. As desvantagens são que alguns clientes podem ficar limitados pelo esqueleto fornecido, que o padrão pode violar o princípio de substituição de Liskov se uma subclasse suprimir uma etapa padrão, e que os métodos template tendem a ser mais difíceis de manter quando têm muitas etapas.

Relação com outros padrões. O Factory Method é uma especialização do Template Method e pode servir como uma de suas etapas. O Template Method é baseado em herança e funciona em nível de classe (é estático), enquanto o Strategy é baseado em composição e funciona em nível de objeto (permite trocar o comportamento em tempo de execução).

4.10 Visitor

Também conhecido como: Visitante.

Intenção. Separar os algoritmos dos objetos sobre os quais eles operam.

Problema. Imagine uma estrutura de objetos de classes diferentes, como um grafo geográfico em que os nós podem ser cidades, indústrias e pontos turísticos. Em um dado momento, surge a necessidade de adicionar uma nova operação, como exportar o grafo para XML. Alterar todas as classes de nós para incluir esse código seria arriscado, pois o código já está em produção e funcionando, e ainda violaria o SRP, misturando a lógica de exportação com a lógica de negócio dos nós.

Solução. O novo comportamento é colocado em uma classe separada, o visitante (visitor), que tem um método para cada tipo de elemento da estrutura. Cada elemento, por sua vez, implementa um método de aceitação (accept), que recebe o visitante e chama nele o método correspondente à sua própria classe. Essa técnica é chamada de despacho duplo (double dispatch) e permite escolher o método certo sem usar condicionais, mesmo em linguagens que não têm sobrecarga de métodos por tipo de objeto.

Estrutura.

Visitante: interface que declara um conjunto de métodos de visita, um para cada classe concreta de elemento.
Visitantes concretos: implementam várias versões do mesmo comportamento, adaptadas a cada classe de elemento. Elemento: interface que declara o método de aceitação, que recebe o visitante. Elementos concretos: implementam o método de aceitação, redirecionando a chamada para o método de visita adequado. Cliente: representa uma coleção ou uma estrutura complexa e normalmente trabalha com ela por uma interface abstrata.

Como implementar. (1) Declarar a interface do visitante, com um método para cada classe concreta de elemento. (2) Declarar a interface do elemento, com o método de aceitação. (3) Implementar o método de aceitação em cada elemento concreto, chamando o método de visita correspondente. (4) Criar um visitante concreto para cada operação nova. (5) Fazer o cliente criar o visitante e passá-lo aos elementos por meio do método de aceitação.

Exemplo de código. O Código 22 calcula a área de formas geométricas diferentes sem colocar esse cálculo dentro das próprias formas.

Código 22 - Exemplo de Visitor em TypeScript

```
interface Visitante {
  visitarCirculo(c: Circulo): number;
  visitarRetangulo(r: Retangulo): number;
}
interface Forma {
  aceitar(v: Visitante): number;
}
class Circulo implements Forma {
  constructor(public raio: number) {}
  aceitar(v: Visitante) { return v.visitarCirculo(this); }
}
class Retangulo implements Forma {
  constructor(public largura: number, public altura: number) {}
  aceitar(v: Visitante) { return v.visitarRetangulo(this); }
}
class VisitanteArea implements Visitante {
  visitarCirculo(c: Circulo) { return Math.PI * c.raio ** 2; }
  visitarRetangulo(r: Retangulo) { return r.largura * r.altura; }
}
const formas: Forma[] = [new Circulo(2), new Retangulo(3, 4)];
const area = new VisitanteArea();
formas.forEach((f) => console.log(f.aceitar(area).toFixed(2))); // 12.57
e 12.00
```

Fonte: elaborado pelo autor (2026).

Quando usar. Quando é preciso executar uma operação em todos os elementos de uma estrutura complexa de objetos, como uma árvore; para separar a lógica de negócio dos comportamentos auxiliares; e quando um comportamento só faz sentido para algumas classes de uma hierarquia, mas não para outras.

Vantagens e desvantagens. O padrão respeita o OCP, já que novos comportamentos podem ser adicionados sem alterar as classes dos elementos, e o SRP, pois várias versões do mesmo comportamento ficam na mesma classe. Também permite acumular informações úteis enquanto se percorre uma estrutura complexa. As desvantagens são a necessidade de atualizar todos os visitantes sempre que uma classe de elemento é adicionada ou removida da hierarquia e a possível falta de acesso aos membros privados dos elementos, o que pode obrigar a quebrar o encapsulamento.

Relação com outros padrões. O Visitor pode ser tratado como uma versão mais poderosa do Command, já que os objetos executam operações sobre vários objetos de classes diferentes. É possível usar o Visitor para executar uma operação sobre uma árvore Composite inteira, e o Iterator pode percorrer a estrutura, enquanto o Visitor executa a operação em cada elemento.

5 CONSIDERAÇÕES FINAIS

Depois de passar pelos 22 padrões, dá para perceber que cada grupo resolve um tipo de problema. Os criacionais ajudam a tirar do código a responsabilidade de decidir como e quais objetos criar. Os estruturais organizam classes e objetos em estruturas maiores sem deixá-las engessadas. Já os comportamentais cuidam de como os objetos se comunicam e de quem faz o quê. Também ficou claro que os padrões se relacionam bastante: o Strategy e o State têm estruturas quase iguais, o Command e o Memento costumam trabalhar juntos no desfazer, o Composite é percorrido por Iterators e Visitors, e o Factory Method muitas vezes é o ponto de partida para chegar ao Abstract Factory ou ao Builder.

Outro ponto que se repete em quase todos os padrões são os mesmos princípios de projeto: programar para interfaces e não para implementações, preferir composição à herança e manter o código aberto para extensão e fechado para modificação. Por isso, estudar os padrões também ajuda a entender melhor a orientação a objetos como um todo.

Por outro lado, os padrões não são uma regra obrigatória, nem um objetivo em si. Usá-los em situações simples só aumenta a complexidade do sistema, e alguns, como o Singleton, são bastante criticados quando usados em excesso. O mais importante é identificar se o problema realmente existe antes de aplicar a solução. Além disso, conhecer os padrões cria um vocabulário comum entre os desenvolvedores: dizer que algo é um Observer ou um Facade comunica a ideia de projeto de forma rápida e precisa, o que facilita o trabalho em equipe.

APÊNDICE A - APLICAÇÕES NO PROJETO ADEGA EXPRESS

Este apêndice apresenta trechos do código-fonte do Adega Express, sistema desenvolvido no âmbito da disciplina, como exemplos práticos relacionados aos padrões de projeto estudados. Os trechos são de autoria do projeto acadêmico e foram examinados por leitura estática do repositório. As aproximações são descritas com rigor: não se afirma que todos os 22 padrões GoF estão implementados no sistema.

A.1 Singleton - instância compartilhada do PrismaClient

No backend, o módulo exporta uma instância compartilhada de PrismaClient. É uma implementação idiomática relacionada ao Singleton, baseada no cache de módulos; não emprega construtor privado e getInstance().

Arquivo: adega-express-backend/src/services/prisma.service.ts

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient({
  log: [],
});
export default prisma;
```

Limitação: O compartilhamento vale por instância de módulo/processo e não impõe uma única instância entre servidores.

A.2 Chain of Responsibility - encadeamento de middlewares

A aplicação compõe autenticação e autorização em sequência antes dos controladores. Isso se aproxima do princípio da cadeia de responsabilidades, embora utilize o mecanismo de middlewares do Express, e não uma hierarquia GoF de manipuladores.

Arquivo: adega-express-backend/src/middlewares/requireRole.middleware.ts

```
export function requireRole(...roles: Role[]) {
  return (req: Request, res: Response, next: NextFunction): void => {
    if (!req.user) {
      res.status(401).json({ status: 'error', message: 'Nao
autenticado.' });
      return;
    }
    if (!roles.includes(req.user.role)) {
      res.status(403).json({
        status: 'error', message: 'Acesso negado: permissao
insuficiente.'
      });
      return;
    }
    next();
  };
}
```

Limitação: O recorte é original do middleware de autorização; o encadeamento completo é configurado nas rotas.

A.3 State - máquina de estados de pedidos

As transições permitidas de pedidos são definidas em uma tabela central. Essa organização ilustra estado e transição, mas não corresponde integralmente ao padrão GoF State, que costuma encapsular comportamentos em objetos de estado.

Arquivo: adega-express-backend/src/lib/business.ts

```
const orderTransitions: Record<OrderStatus, OrderStatus[]> = {
  PENDENTE: ['CONFIRMADO', 'CANCELADO'],
  CONFIRMADO: ['SEPARADO', 'CANCELADO'],
  SEPARADO: ['EM_ROTA', 'CANCELADO'],
  EM_ROTA: ['ENTREGUE'],
  ENTREGUE: [],
  CANCELADO: [],
};
```

Limitação: O código exemplifica uma máquina de estados, não a estrutura clássica por polimorfismo de estados.

REFERÊNCIAS

ADEGA EXPRESS. Repositório do projeto acadêmico. GitHub. Disponível em: <https://github.com/gustavinzx/AdegaExpress>. Acesso em: 10 nov. 2026.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Padrões de projeto: soluções reutilizáveis de software orientado a objetos**. Porto Alegre: Bookman, 2000.

REFACTORING.GURU. **O catálogo dos padrões de projeto**. Disponível em: <https://refactoring.guru/pt-br/design-patterns/catalog>. Acesso em: 10 nov. 2026.